

Computer Vision:

Homework-2

Bilal Ahmed
ahmedb@purdue.edu

Homography

Homography or planar projective transform refers to a mapping that transforms points on a plane (e.g. 2D surface) to another plane. It is a powerful concept that makes use of homogeneous coordinate (HC) representations and can also be used to project one image onto another one. For example, say we have an image frame and we want to project a new image onto that frame, homography does that very easily for us. For mapping from one 2D plane to another 2D plane, homography would be 3×3 matrix H .

Let's take a points $(x, y) \in \mathbb{R}^2$ on one plane and we want to transform this to a point $(x', y') \in \mathbb{R}^2$ on another plane. We can do that using homogeneous coordinate representation of these points and

homography matrix. HC representations of these points would be $\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$ and $\begin{bmatrix} x'_1 \\ x'_2 \\ x'_3 \end{bmatrix}$ respectively.

$$\begin{bmatrix} x'_1 \\ x'_2 \\ x'_3 \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

Writing the matrix-vector multiplication in the algebraic form;

$$x'_1 = h_{11}x_1 + h_{12}x_2 + h_{13}x_3$$

$$x'_2 = h_{21}x_1 + h_{22}x_2 + h_{23}x_3$$

$$x'_3 = h_{31}x_1 + h_{32}x_2 + h_{33}x_3$$

Let's get the expression for the physical coordinates (x', y') , using $x' = \frac{x'_1}{x'_3}$ and $y' = \frac{x'_2}{x'_3}$. So we have,

$$x' = \frac{x'_1}{x'_3} = \frac{h_{11}x_1 + h_{12}x_2 + h_{13}x_3}{h_{31}x_1 + h_{32}x_2 + h_{33}x_3}$$

and

$$y' = \frac{x'_2}{x'_3} = \frac{h_{21}x_1 + h_{22}x_2 + h_{23}x_3}{h_{31}x_1 + h_{32}x_2 + h_{33}x_3}$$

Dividing numerator and denominator of both the equations by x_3 gives us physical coordinates (x, y) on the right hand side as well.

$$x' = \frac{h_{11}x + h_{12}y + h_{13}}{h_{31}x + h_{32}y + h_{33}}$$

$$y' = \frac{h_{21}x + h_{22}y + h_{23}}{h_{31}x + h_{32}y + h_{33}}$$

Rearranging,

$$h_{11}x + h_{12}y + h_{13} - h_{31}xx' - h_{32}yx' - h_{33}x' = 0$$

$$h_{21}x + h_{22}y + h_{23} - h_{31}xy' - h_{32}yy' - h_{33}y' = 0$$

Since H is also homogeneous, meaning what matters is the ratios of its components, we can set $h_{33} = 1$. In that case the above expressions become;

$$h_{11}x + h_{12}y + h_{13} - h_{31}xx' - h_{32}yx' = x'$$

$$h_{21}x + h_{22}y + h_{23} - h_{31}xy' - h_{32}yy' = y'$$

We can also think of the previous step as dividing everything by h_{33} and learning the 8 ratios. In that case, the parameters we'll be learning would be (e.g.) $h'_{11} = \frac{h_{11}}{h_{33}}$. But since H is homogeneous, learning these 8 ratios would be enough. To avoid complicating the matters, instead of expressing as ratios, we can simply think of it as setting $h_{33} = 1$.

A pair of corresponding points in both planes gives us 2 equations, since we need to learn 8 parameters, we'd need 4 such pairs. Using 4 pairs of points, we'd have 8 equations;

$$h_{11}x_1 + h_{12}y_1 + h_{13} - h_{31}x_1x'_1 - h_{32}y_1x'_1 = x'_1$$

$$h_{21}x_1 + h_{22}y_1 + h_{23} - h_{31}x_1y'_1 - h_{32}y_1y'_1 = y'_1$$

$$h_{11}x_2 + h_{12}y_2 + h_{13} - h_{31}x_2x'_2 - h_{32}y_2x'_2 = x'_2$$

$$h_{21}x_2 + h_{22}y_2 + h_{23} - h_{31}x_2y'_2 - h_{32}y_2y'_2 = y'_2$$

$$h_{11}x_3 + h_{12}y_3 + h_{13} - h_{31}x_3x'_3 - h_{32}y_3x'_3 = x'_3$$

$$h_{21}x_3 + h_{22}y_3 + h_{23} - h_{31}x_3y'_3 - h_{32}y_3y'_3 = y'_3$$

$$h_{11}x_4 + h_{12}y_4 + h_{13} - h_{31}x_4x'_4 - h_{32}y_4x'_4 = x'_4$$

$$h_{21}x_4 + h_{22}y_4 + h_{23} - h_{31}x_4y'_4 - h_{32}y_4y'_4 = y'_4$$

We can write these 8 set of equations in the matrix form;

$$\begin{bmatrix} x_1 & y_1 & 1 & 0 & 0 & 0 & -x_1x'_1 & -y_1x'_1 \\ 0 & 0 & 0 & x_1 & y_1 & 1 & -x_1y'_1 & -y_1y'_1 \\ x_2 & y_2 & 1 & 0 & 0 & 0 & -x_2x'_2 & -y_2x'_2 \\ 0 & 0 & 0 & x_2 & y_2 & 1 & -x_2y'_2 & -y_2y'_2 \\ x_3 & y_3 & 1 & 0 & 0 & 0 & -x_3x'_3 & -y_3x'_3 \\ 0 & 0 & 0 & x_3 & y_3 & 1 & -x_3y'_3 & -y_3y'_3 \\ x_4 & y_4 & 1 & 0 & 0 & 0 & -x_4x'_4 & -y_4x'_4 \\ 0 & 0 & 0 & x_4 & y_4 & 1 & -x_4y'_4 & -y_4y'_4 \end{bmatrix} \begin{bmatrix} h_{11} \\ h_{12} \\ h_{13} \\ h_{21} \\ h_{22} \\ h_{23} \\ h_{31} \\ h_{32} \end{bmatrix} = \begin{bmatrix} x'_1 \\ y'_1 \\ x'_2 \\ y'_2 \\ x'_3 \\ y'_3 \\ x'_4 \\ y'_4 \end{bmatrix}$$

or we can simply write this as;

$$Ax = y$$

So, given 4 corresponding pairs of points, we can solve this set of linear equations using `np.linalg.lstsq`, to get x . We can append 1 as the ninth element and arrange them as 3×3 matrix H .

Affine Homography

Affine homography is a subset (sub-group) of general homography that is characterized by preservation of parrallel lines. For affine homography, matrix H has the following form;

$$H = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ 0 & 0 & 1 \end{bmatrix}$$

So we have 6 parameters to learn, instead of 8. Setting $h_{31} = h_{32} = 0$ in the expressions derived for the general case, we have;

$$\begin{bmatrix} x_1 & y_1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & x_1 & y_1 & 1 \\ x_2 & y_2 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & x_2 & y_2 & 1 \\ x_3 & y_3 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & x_3 & y_3 & 1 \\ x_4 & y_4 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & x_4 & y_4 & 1 \end{bmatrix} \begin{bmatrix} h_{11} \\ h_{12} \\ h_{13} \\ h_{21} \\ h_{22} \\ h_{23} \end{bmatrix} = \begin{bmatrix} x'_1 \\ y'_1 \\ x'_2 \\ y'_2 \\ x'_3 \\ y'_3 \\ x'_4 \\ y'_4 \end{bmatrix}$$

For 6 parameters, we should be able to use 3 pairs of corresponding points, but let's solve using 4 pairs of points, since we already have 4 points. This makes it an over-determined system.

Task-1

I use the 4 corners of the photo-frame (the black portion) in the given views as PQRS region and almost the entire image of Alex Honnold as ROI. For nothing the coordinates, I plotted the images using matplotlib and manually noted down the coordinates of 3 corners.

Projecting image using general homography

In the first task, we will use homography to project an image of Alex Honnold onto a section of another image. Here is the step by step procedure, I used;

- Note down the coordinates of four corners of the section that needs to be updated.
- Note down 4 corresponding corners of the region of interest (ROI) in the Alex Honnold image.

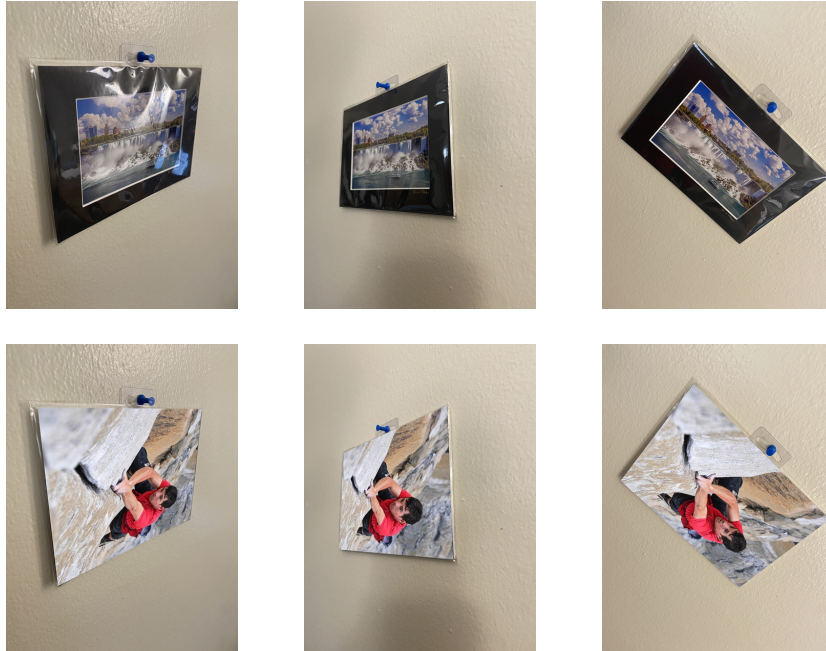


Figure 1: Top row shows original frame images and bottom row shows corresponding images with image of Alex Honnold projected onto the frames

- I compute homography H that maps from coordinates in photo-frame image to the image of interest.
- Using the computed H , for each coordinate within the photo-frame, I get the corresponding coordinates in the image of interest, pick its pixel values and write them back to the photo-frame.

Fig. 1 shows the 3 frame images in the top row and corresponding images with image of Alex Honnold projected onto the frames, in the bottom row.

Combining homographies

As in the previous section, we saw that in order to project fig. a onto fig. b, we compute homography that maps from fig. b back to fig. a and then for every pixel in fig. b we pick pixel values from fig. a. I use the same approach for this task as well. Here are the steps;

- Compute H_1 that maps from fig. 1b to fig. 1a.
- Compute H_2 that maps from fig. 1c to fig. 1b.
- Then I compute $H = H_1 \cdot H_2$, which would map each pixel in fig. 1c to fig. 1a.
- Then for the PQRS area in fig. 1c, I pick the corresponding pixel values from the PQRS area in fig. 1a using the combined homography H .

As a result we get area PQRS of fig. 1a projected onto PQRS of fig. 1c as shown in Fig. 2.



Figure 2: On left is the original fig. 1a and on the right is the fig. 1a projected onto the frame of fig. 1c

Using affine homography

Doing the task 1.1 using affine homography. Fig.reffig:affine-h shows the result of using affine homography. Comparing this to the earlier results obtained using general homography, this is not that good. Using affine homography misses the corners and it seems like it tries to maintain parallel lines while projecting, which makes sense because that is the feature of affine homography.

Task-2

The image of interest that I am going to project onto different views of my monitor is a pony image, shown in fig. 4, Repeating all the steps using figures captured by my phone. I will be using an image a pony as image of interest and project it onto different views of my monitor. The coordinates of 4 corners of the monitor are highlighted as red circles in the original images (top row). Fig. 5 shows results of sub task 2.1

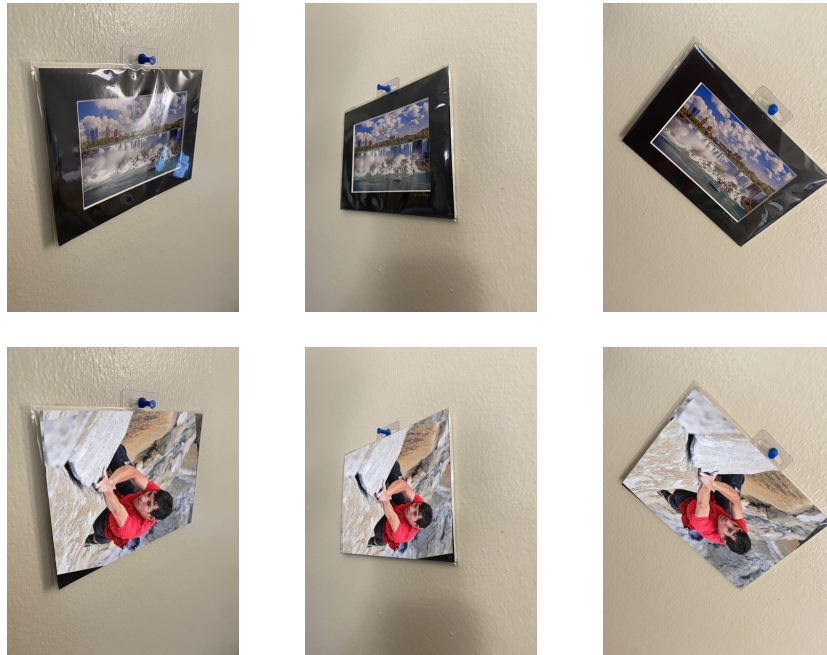


Figure 3: Projections using affine homography



Figure 4: Image of a pony

Combining homographies

This section is where I project view 1 of my monitor (shown top left of fig. 5) onto view 3 (shown top right of fig. 5). Note that only view 1 has error message displayed in the middle. So it can be distinguished from the view 2 and 3. Fig. 6 shows the output for this task.

Using affine homography



Figure 5: Top row shows original frame images and bottom row shows corresponding images with image of a pony projected onto the frames

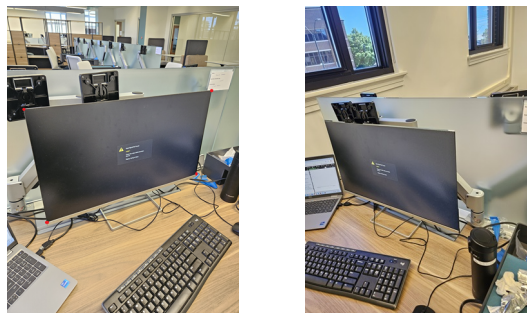


Figure 6: On left is the original fig. 2a and on the right is the fig. 2a projected onto the frame of fig. 2c

Python Code

```

1  import cv2
2  import pathlib
3  import skimage
4  import computer_vision
5
6  import matplotlib.pyplot as plt
7  import matplotlib as mpl
8  import numpy as np
9
10 def compute_affine_homography(point_pairs):
11     """Takes 3 pairs of points in two images, where
12     each pair of points is a pixel location in one image and corresponding
13     pixel location in the 2nd image.

```



Figure 7: Projections of pony onto 2a, 2b and 2c using affine homography

```

14  Computes general planar projective transform that maps pixel locations
15  in the image 1 to pixel locations in the image 2.
16
17  Args:
18      point_pairs: list(list(tuples)): Each pair of point is represented
19                  as [(x,y), (x', y')]. Function expects 4 such pairs of points.
20  """
21  num_points = len(point_pairs)
22  assert (num_points) ==4, f"expects 4 pairs of points but got {num_points}"
23
24  (x1, y1), (x_p1, y_p1) = point_pairs[0]
25  (x2, y2), (x_p2, y_p2) = point_pairs[1]
26  (x3, y3), (x_p3, y_p3) = point_pairs[2]
27  (x4, y4), (x_p4, y_p4) = point_pairs[3]
28
29
30  A = np.array([
31      [x1, y1, 1, 0, 0, 0],
32      [0, 0, 0, x1, y1, 1],
33      [x2, y2, 1, 0, 0, 0],
34      [0, 0, 0, x2, y2, 1],
35      [x3, y3, 1, 0, 0, 0],
36      [0, 0, 0, x3, y3, 1],
37      [x4, y4, 1, 0, 0, 0],
38      [0, 0, 0, x4, y4, 1],
39  ])
40  # y needs to be column vector
41  y = np.array([x_p1, y_p1, x_p2, y_p2, x_p3, y_p3, x_p4, y_p4])[:, None]
42
43  h = np.linalg.lstsq(A, y)[0]
44  # homography matrix would be...
45  H = h.reshape(2,3)

```

```

45     H = np.concatenate([H, np.array([0, 0, 1])[None,:]], axis=0)
46     return H
47
48
49 def compute_general_homography(point_pairs):
50     """Takes 4 pairs of points in two images, where
51     each pair of points is a pixel location in one image and corresponding
52     pixel location in the 2nd image.
53     Computes general planar projective transform that maps pixel locations
54     in the image 1 to pixel locations in the image 2.
55
56     Args:
57         point_pairs: list(list(tuples)): Each pair of point is represented
58                     as [(x,y), (x', y')]. Function expects 4 such pairs of points.
59     """
60     num_points = len(point_pairs)
61     assert (num_points) ==4, f"expects 4 pairs of points but got {num_points}"
62
63     (x1, y1), (x_p1, y_p1) = point_pairs[0]
64     (x2, y2), (x_p2, y_p2) = point_pairs[1]
65     (x3, y3), (x_p3, y_p3) = point_pairs[2]
66     (x4, y4), (x_p4, y_p4) = point_pairs[3]
67
68     A = np.array([
69         [x1, y1, 1, 0, 0, 0, -x1*x_p1, -y1*x_p1],
70         [0, 0, 0, x1, y1, 1, -x1*y_p1, -y1*y_p1],
71         [x2, y2, 1, 0, 0, 0, -x2*x_p2, -y2*x_p2],
72         [0, 0, 0, x2, y2, 1, -x2*y_p2, -y2*y_p2],
73         [x3, y3, 1, 0, 0, 0, -x3*x_p3, -y3*x_p3],
74         [0, 0, 0, x3, y3, 1, -x3*y_p3, -y3*y_p3],
75         [x4, y4, 1, 0, 0, 0, -x4*x_p4, -y4*x_p4],
76         [0, 0, 0, x4, y4, 1, -x4*y_p4, -y4*y_p4],
77     ])
78     # y needs to be column vector
79     y = np.array([x_p1, y_p1, x_p2, y_p2, x_p3, y_p3, x_p4, y_p4])[:, None]
80
81     h = np.linalg.lstsq(A, y)[0]
82     h = np.append(h, 1)
83     # homography matrix would be...
84     H = h.reshape(3,3)
85     return H
86
87 def transform_pixel_coordinate(H, pixel_coordinates):
88     """Applies homography H to the 2D pixel coordinates
89     and returns transformed 2D pixel coordinates.
90
91     Args:
92         H: shape (3, 3): Homography matrix
93         pixel_coordinates: shape (2,): either a tuple or a list.
94     """
95     pixel_hc = np.array([*pixel_coordinates, 1])
96     transformed_hc = np.dot(H, pixel_hc)
97     transformed_hc /= transformed_hc[-1]
98     return transformed_hc[:-1]
99
100 def project_image_onto_frame(frame_image, image_of_interest, points_on_frame,
101                             points_on_image,
102                             affine_homography=False):

```

```

102     """Given 4 points on the frame image and 4 corresponding points on the
103         image of interest,
104     projects region of interest (ROI) from image of interest (based on
105         coordinates provided),
106     onto the frame.
107
108     Args:
109         frame_image: frame image
110         image_of_interest: image of interest
111         points_on_frame: shape [(x,y), (x,y), (x,y), (x,y)] = corner points
112             PQRS of the frame
113         points_on_image: shape [(x,y), (x,y), (x,y), (x,y)] = corner points
114             defining ROI on the image.
115
116     Returns:
117         modified_frame_image:
118     """
119     pairs_of_points = []
120     for p1, p2 in zip(points_on_frame, points_on_image):
121         pairs_of_points.append([p1, p2])
122
123     # compute homography that gives pixel coordinates on the image for
124     # given pixel coordinate on the frame.
125     if affine_homography:
126         # needs only 3 points for affine...
127         H = compute_affine_homography(pairs_of_points)
128     else:
129         H = compute_general_homography(pairs_of_points)
130
131     # read pixels of the frame as a polygon
132     rows = [x for x, _ in points_on_frame]
133     cols = [y for _, y in points_on_frame]
134     rr, cc = skimage.draw.polygon(rows, cols)
135
136     # update pixels within the polygon by the corresponding pixels
137     # from the image of interest...
138     for r, c in zip(rr, cc):
139         rt, ct = transform_pixel_coordinate(H, np.array([r, c]))
140         if rt < image_of_interest.shape[0] and ct < image_of_interest.shape
141             [1]:
142             pixel_value = image_of_interest[int(rt), int(ct)]
143             frame_image[r, c] = pixel_value
144
145     return frame_image
146
147 def read_image(subdir, image_name):
148     """Reads images from the subdir"""
149     path = pathlib.Path(computer_vision.__path__[0])
150     image_path = path.parents[0] / 'images' / subdir / image_name
151     img_bgr = cv2.imread(image_path)
152     # Convert the image from BGR to RGB format
153     img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
154     return img_rgb
155
156 def save_image(image, subdir, image_name):
157     """Saves image to the subdir"""
158     path = pathlib.Path(computer_vision.__path__[0])
159     image_path = path.parents[0] / 'images' / subdir / image_name
160     # plt.imshow(image)

```

```
156 # plt.savefig(image_path)
157 # Save the image with specific quality
158 img_bgr_for_saving = cv2.cvtColor(image, cv2.COLOR_RGB2BGR)
159 cv2.imwrite(image_path, img_bgr_for_saving)
160
161
162 # Task: 1.1: project 1d onto 1a
163 P = (830, 410)
164 Q = (850, 2550)
165 R = (2288, 2400)
166 S = (3170, 675)
167
168 P_roi = (0, 0)
169 Q_roi = (0, 780)
170 R_roi = (660, 780)
171 S_roi = (660, 0)
172
173
174 points_on_frame = [P, Q, R, S]
175 points_on_image = [P_roi, Q_roi, R_roi, S_roi]
176
177 frame_image = read_image(subdir='hw2', image_name='img1.jpg')
178 image_of_interest = read_image(subdir='hw2', image_name='alex_honnold.jpg')
179
180 updated_frame_image = project_image_onto_frame(
181     frame_image=frame_image,
182     image_of_interest=image_of_interest,
183     points_on_frame=points_on_frame,
184     points_on_image=points_on_image
185 )
186
187 save_image(updated_frame_image, 'hw2', 'projected-image-1a.jpg')
188
189 # Task: 1.1: project 1d onto 1b
190 # corners of frame
191 P = (1425, 518)
192 Q = (802, 1861)
193 R = (2820, 1946)
194 S = (2690, 456)
195
196 # ROI on the image of interest
197 P_roi = (0, 0)
198 Q_roi = (0, 780)
199 R_roi = (660, 780)
200 S_roi = (660, 0)
201
202
203 points_on_frame = [P, Q, R, S]
204 points_on_image = [P_roi, Q_roi, R_roi, S_roi]
205
206 frame_image = read_image(subdir='hw2', image_name='img2.jpg')
207 image_of_interest = read_image(subdir='hw2', image_name='alex_honnold.jpg')
208
209 updated_frame_image = project_image_onto_frame(
210     frame_image=frame_image,
211     image_of_interest=image_of_interest,
212     points_on_frame=points_on_frame,
213     points_on_image=points_on_image
214 )
```

```
215 save_image(updated_frame_image, 'hw2', 'projected-image-1b.jpg')
216
217 # Task: 1.1: project 1d onto 1c
218 # corners of frame
219 P = (520, 1201)
220 Q = (2295, 2980)
221 R = (3190, 1794)
222 S = (1792, 242)
223
224 # ROI on the image of interest
225 P_roi = (0, 0)
226 Q_roi = (0, 780)
227 R_roi = (660, 780)
228 S_roi = (660, 0)
229
230
231 points_on_frame = [P, Q, R, S]
232 points_on_image = [P_roi, Q_roi, R_roi, S_roi]
233
234 frame_image = read_image(subdir='hw2', image_name='img3.jpg')
235 image_of_interest = read_image(subdir='hw2', image_name='alex_honnold.jpg')
236
237 updated_frame_image = project_image_onto_frame(
238     frame_image=frame_image,
239     image_of_interest=image_of_interest,
240     points_on_frame=points_on_frame,
241     points_on_image=points_on_image
242 )
243
244 save_image(updated_frame_image, 'hw2', 'projected-image-1c.jpg')
245
246 # Task: 1.2: combining homographies
247 # coordinates on img1
248 P = (830, 410)
249 Q = (850, 2550)
250 R = (2288, 2400)
251 S = (3170, 675)
252 points_on_1a = [P, Q, R, S]
253
254 # coordinates on img2
255 # corners of frame
256 P = (1425, 518)
257 Q = (802, 1861)
258 R = (2820, 1946)
259 S = (2690, 456)
260 points_on_1b = [P, Q, R, S]
261
262 # coordinates on img3
263 # corners of frame
264 P = (520, 1201)
265 Q = (2295, 2980)
266 R = (3190, 1794)
267 S = (1792, 242)
268
269 points_on_1c = [P, Q, R, S]
270
271 # compute homography that takes from 1b to 1a
272 point_pairs = [[p1, p2] for p1, p2 in zip(points_on_1b, points_on_1a)]
273
```

```
274 H1 = compute_general_homography(point_pairs)
275
276 # compute homography that takes from 1c to 1b
277 point_pairs = [[p1, p2] for p1, p2 in zip(points_on_1c, points_on_1b)]
278 H2 = compute_general_homography(point_pairs)
279
280 # product of 2 homographies..
281 H = np.matmul(H1, H2)
282
283 # get coordinates of target frame
284 rows = [x for x, _ in points_on_1c]
285 cols = [y for _, y in points_on_1c]
286 rr, cc = skimage.draw.polygon(rows, cols)
287
288 frame_image = read_image(subdir='hw2', image_name='img3.jpg')
289 image_of_interest = read_image(subdir='hw2', image_name='img1.jpg')
290
291 # update pixels within the polygon by the corresponding pixels
292 # from the image of interest...
293 for r, c in zip(rr, cc):
294     rt, ct = transform_pixel_coordinate(H, np.array([r, c]))
295     pixel_value = image_of_interest[int(rt), int(ct)]
296     frame_image[r, c] = pixel_value
297
298 # plt.imshow(updated_frame_image)
299 save_image(frame_image, 'hw2', '1a-transformed-to-1c.jpg')
300
301
302 # Task 1.3: Affine Homography: 1a
303 P = (830, 410)
304 Q = (850, 2550)
305 R = (2288, 2400)
306 S = (3170, 675)
307
308 P_roi = (0, 0)
309 Q_roi = (0, 780)
310 R_roi = (660, 780)
311 S_roi = (660, 0)
312
313 points_on_frame = [P, Q, R, S]
314 points_on_image = [P_roi, Q_roi, R_roi, S_roi]
315
316 frame_image = read_image(subdir='hw2', image_name='img1.jpg')
317 image_of_interest = read_image(subdir='hw2', image_name='alex_honnold.jpg')
318
319 updated_frame_image = project_image_onto_frame(
320     frame_image=frame_image,
321     image_of_interest=image_of_interest,
322     points_on_frame=points_on_frame,
323     points_on_image=points_on_image,
324     affine_homography=True
325 )
326
327 save_image(updated_frame_image, 'hw2', 'projected-image-1a-using-affine.jpg')
328
329 # Task 1.3: Affine Homography: 1b
330 # corners of frame
331 P = (1425, 518)
```

```
332 Q = (802, 1861)
333 R = (2820, 1946)
334 S = (2690, 456)
335
336 P_roi = (0, 0)
337 Q_roi = (0, 780)
338 R_roi = (660, 780)
339 S_roi = (660, 0)
340
341
342 points_on_frame = [P, Q, R, S]
343 points_on_image = [P_roi, Q_roi, R_roi, S_roi]
344
345 frame_image = read_image(subdir='hw2', image_name='img2.jpg')
346 image_of_interest = read_image(subdir='hw2', image_name='alex_honnold.jpg')
347
348 updated_frame_image = project_image_onto_frame(
349     frame_image=frame_image,
350     image_of_interest=image_of_interest,
351     points_on_frame=points_on_frame,
352     points_on_image=points_on_image,
353     affine_homography=True
354 )
355
356 save_image(updated_frame_image, 'hw2', 'projected-image-1b-using-affine.jpg')
357
358
359 # Task 1.3: Affine Homography: 1c
360 # corners of frame
361 P = (520, 1201)
362 Q = (2295, 2980)
363 R = (3190, 1794)
364 S = (1792, 242)
365
366 P_roi = (0, 0)
367 Q_roi = (0, 780)
368 R_roi = (660, 780)
369 S_roi = (660, 0)
370
371
372 points_on_frame = [P, Q, R, S]
373 points_on_image = [P_roi, Q_roi, R_roi, S_roi]
374
375 frame_image = read_image(subdir='hw2', image_name='img3.jpg')
376 image_of_interest = read_image(subdir='hw2', image_name='alex_honnold.jpg')
377
378 updated_frame_image = project_image_onto_frame(
379     frame_image=frame_image,
380     image_of_interest=image_of_interest,
381     points_on_frame=points_on_frame,
382     points_on_image=points_on_image,
383     affine_homography=True
384 )
385
386 save_image(updated_frame_image, 'hw2', 'projected-image-1c-using-affine.jpg')
```

Listing 1: Example Python code